TP 4 : Algorithmes de tri

1. Algorithmes de tri fondamentaux

1. Implémenter les algorithmes de tri simples vus en cours : *tri par sélection*, *tri par insertion* et *tri à bulles*.
2. Générer aléatoirement plusieurs tableaux en variant leurs tailles (100, 1 000, 10 000, etc.) et comparer le temps d'exécution des trois algorithmes.
3. Quel algorithme s'avère être le plus rapide ?
4. Écrire un algorithme de tri de piles fondé sur le principe du tri par sélection.
   Contrainte : La pile est la seule structure de données autorisée (ni files, ni listes).
5. Le tri Gnome est un algorithme inspiré de la façon dont un lutin de jardin aligne des pots de fleurs par taille : il regarde le pot courant et le précédent, les échange s'ils sont dans le mauvais ordre et recule d'un pas, sinon il avance d'un pas.
   — Écrire l'algorithme du tri Gnome. Quelle est sa complexité théorique dans le pire des cas et dans le meilleur des cas ? À quel tri classique ressemble sa mécanique de recul ?

2. Tris récursifs et optimisations

6. Reprendre le *tri fusion* vu en cours. Générer aléatoirement des listes en variant leurs tailles et comparer les temps d'exécution entre elles, puis par rapport aux tris de la première partie.
7. Le *tri par insertion* est généralement plus efficace que le tri fusion sur de petites listes. Écrire une version optimisée du *tri fusion* qui bascule sur le tri par insertion lorsque la taille de la sous-liste est inférieure à un seuil prédéfini.
8. Le *tri mixte* consiste à diviser une liste en k sous-listes, à trier chacune d'elles séparément (par exemple avec un tri par insertion), puis à les fusionner. Implémenter ce tri et déterminer empiriquement la valeur de k offrant la meilleure complexité.

3. Variantes et extensions

9. Réaliser avec l'algorithme de votre choix une fonction `trie_chaine` de type $string \to string$ qui renvoie la chaîne de caractères triée par ordre alphabétique.
10. Améliorer le *tri par insertion* en effectuant la recherche de la position d'insertion à l'aide d'une **recherche dichotomique**.
11. Implémenter le *tri rapide* (*quicksort*), puis comparer ses performances avec celles du tri fusion sur des listes aléatoires.
12. Tester différentes stratégies pour le choix du pivot dans le tri rapide (premier élément, élément médian, choix aléatoire).
13. Écrire une version récursive de la fonction `partition` du tri rapide.

4. Tri et structures de données

1. Écrire un programme permettant de trier par ordre croissant une liste de points du plan. Un point $P(x, y)$ est défini par son abscisse et son ordonnée. Un point est considéré comme inférieur à un autre si sa **distance à l'origine** est plus petite. Vous devez implémenter au moins deux algorithmes de tri distincts parmi ceux étudiés.